UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL RESISTENCIA (UTN FRRe)
Tecnicatura Universitaria en Logística
Extensión Ituzaingó (Corrientes)

Informática Aplicada
Prof. Ing. Juan Manuel Goya

Trabajo práctico: del problema a la base de datos

Etapa 1 — Empresa de bicicletas compartidas

Alumno: Santiago T. Sala
Legajo: 30794
Fecha: 12 de octubre de 2026

Repositorio del trabajo: https://github.com/jumptowin/informatica-aplicada-tul (carpeta etapa1).

1. Análisis del enunciado

El problema. La empresa presta bicicletas que se retiran y se devuelven en estaciones repartidas por la ciudad. Necesita saber en todo momento dónde está cada bicicleta y en qué estado, quién la usó, desde qué estación salió y a cuál llegó, con qué tarjeta pagó, y qué tareas de mantenimiento se hacen en cada estación y quién las hace. Mirado desde la logística, es una red de puntos (las estaciones) con lugar limitado, por la que circulan las bicicletas. Las bicis se gastan y se desacomodan: algunas estaciones se vacían y otras se llenan.

1.1 Entidades, atributos e identificadores

Entidad	Tipo	Identificador	Otros atributos
ESTACION	fuerte	codigo	direccion, capacidad_max
BICICLETA	fuerte	numero_serie	estado (DISPONIBLE / EN_USO / EN_MANTENIMIENTO)
USUARIO	fuerte	dni	nombre
TARJETA (medio de pago)	fuerte	id_tarjeta (código interno)	marca, ultimos_4_digitos, vencimiento
VIAJE	débil (depende de USUARIO y BICICLETA)	usuario + bicicleta + fecha_hora_inicio (clave parcial)	fecha_hora_fin
MANTENIMIENTO (tarea)	fuerte	id_mantenimiento (código interno)	fecha, descripcion
OPERARIO TÉCNICO	fuerte	legajo	nombre

1.2 Relaciones y cardinalidades

Relación (verbo)	Entidades	Tipo	Cómo la leo
posee	USUARIO – TARJETA	1:N	Un usuario tiene 1 o más tarjetas; cada tarjeta es de un solo usuario
realiza (identificadora)	USUARIO – VIAJE	1:N	Un usuario puede tener ninguno o muchos viajes; cada viaje es de un solo usuario
se usa en (identificadora)	BICICLETA – VIAJE	1:N	Una bici puede tener ninguno o muchos viajes; cada viaje usa una sola bici
sale de (origen)	ESTACION – VIAJE	1:N	Cada viaje sale de una sola estación
llega a (destino)	ESTACION – VIAJE	1:N	Cada viaje llega a una estación (ninguna mientras está en curso)
está ubicada en	ESTACION – BICICLETA	1:N	Cada bici pertenece siempre a una estación; una estación tiene de 0 a N bicis (hasta su capacidad)
requiere		1:N	

Relación (verbo)	Entidades	Tipo	Cómo la leo
	ESTACION – MANTENIMIENTO		Una estación puede tener ninguna o muchas tareas; cada tarea es de una estación
realiza	OPERARIO – MANTENIMIENTO	1:N	Un operario hace ninguna o muchas tareas; cada tarea la hace un solo operario
se paga con (la agregué)	TARJETA – VIAJE	1:N	Cada viaje se paga con una tarjeta del mismo usuario

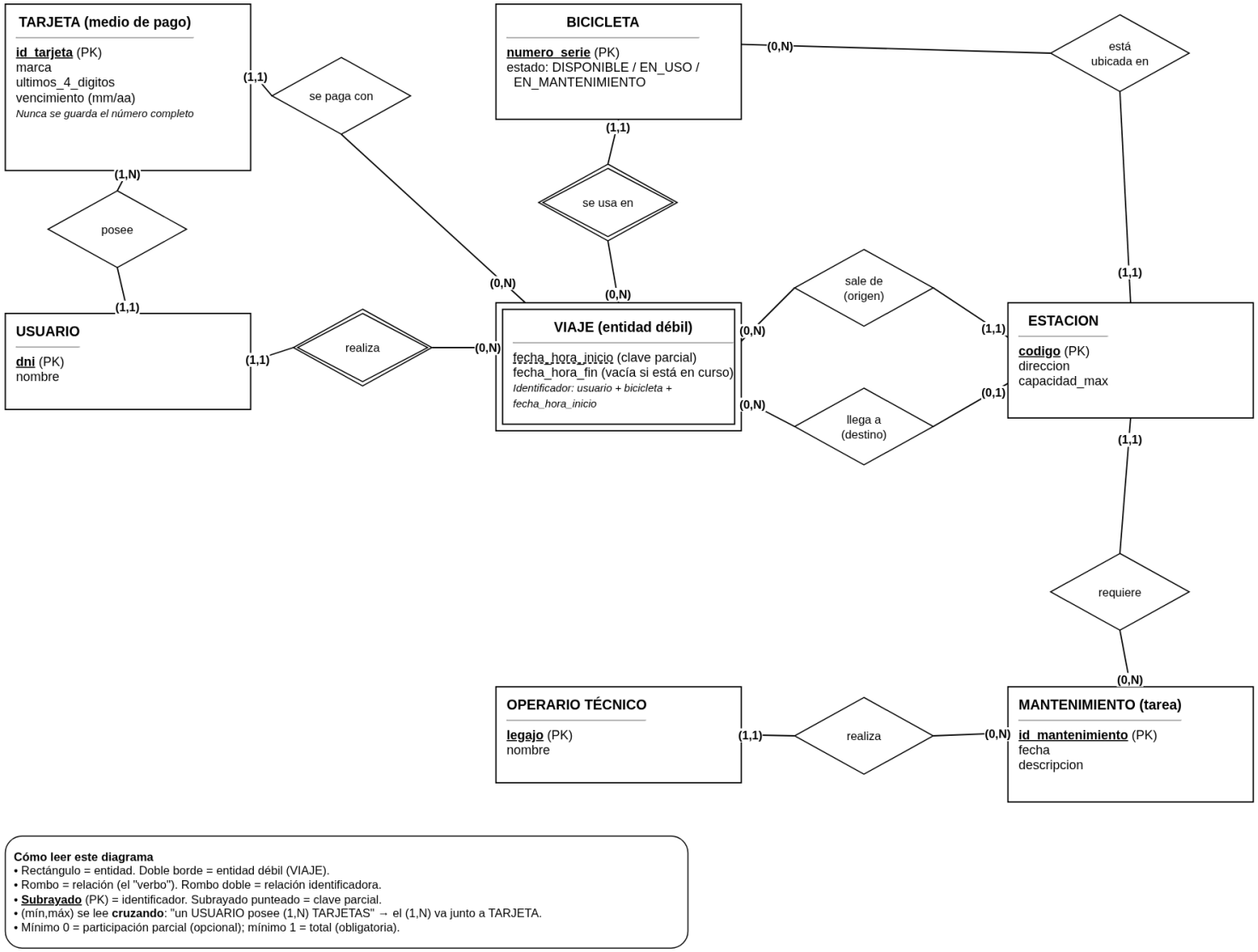
1.3 Puntos dudosos y decisiones que tomé

#	Punto del enunciado	Qué decidí y por qué
1	Dice que la bicicleta “pertenece siempre a una estación”, pero mientras se usa está en la calle.	La bici queda asignada a la última estación (la de origen) hasta que se devuelve, y al cerrar el viaje pasa a la de destino. Así se cumple el “siempre”, y el estado EN_USO indica que no está físicamente ahí.
2	La estación de destino y la hora de fin no se conocen al empezar el viaje.	Las dejo vacías mientras el viaje está en curso. Puse como regla que estén las dos o ninguna.
3	“Un viaje no existe sin la bicicleta ni sin el usuario”.	Por eso tomé VIAJE como entidad débil de las dos. Se identifica con usuario + bicicleta + fecha y hora de inicio.
4	No dice qué datos de la tarjeta se guardan.	Por seguridad no guardo el número completo: solo marca, últimos 4 dígitos y vencimiento, con un código interno.
5	No dice con qué tarjeta se paga cada viaje.	Al armar el diagrama logístico vi que el paso “cobra” necesita ese dato, así que corregí el MER y agregué la relación “se paga con” (la tarjeta tiene que ser del mismo usuario).
6	El mantenimiento es de la estación; el enunciado no lo une a una bicicleta.	En esta etapa lo dejé solo unido a la estación, como dice el texto. La unión con la bicicleta la agregué en la Etapa 2.
7	El estado de la bici tendría que coincidir con los viajes abiertos.	Puse como regla que una bici, o un usuario, no pueda tener dos viajes abiertos al mismo tiempo. El cambio automático de estado lo tendría que hacer la aplicación (Etapa 2).
8	Capacidad máxima de la estación.	Agregué una regla en la base para que no se carguen más bicis que la capacidad.
9	“Uno o más” medios de pago.	En el MER queda como (1,N). En SQL no se puede exigir con una regla simple, así que lo tendría que controlar la aplicación al registrar al usuario.

2. Diagrama MER

Usé la notación de Chen con (mín,máx), leída “cruzando”, como en el ejemplo DEPARTAMENTO (1,1) – EMPLEADO (0,N) de la clase S04. El mínimo indica la participación: 0 es parcial (opcional) y 1 es total (obligatoria). No dibujé las claves foráneas porque aparecen recién en el modelo relacional (punto 4).

MER — Bicicletas compartidas (Etapa 1)



Cómo leer este diagrama
- Rectángulo = entidad. Doble borde = entidad débil (VIAJE).
- Rombo = relación (el "verbo"). Rombo doble = relación identificadora.
- **Subrayado** (PK) = identificador. Subrayado punteado = clave parcial.
- (mín,máx) se lee **cruzando**: "un USUARIO posee (1,N) TARJETAS" → el (1,N) va junto a TARJETA.
- Mínimo 0 = participación parcial (opcional); mínimo 1 = total (obligatoria).

Figura 1. MER de la Etapa 1 (archivo editable: bicicletas_MER.drawio)

3. Diagrama logístico

En el diagrama separé cuatro actores (carriles): Usuario, Sistema/App, Estación (anclajes) y Operario técnico. La bicicleta pasa por estos estados: DISPONIBLE → EN_USO → DISPONIBLE, o EN_USO → EN_MANTENIMIENTO → DISPONIBLE (o fuera de servicio si no tiene arreglo). Marqué con recuadro punteado los dos cuellos de botella que encontré: la estación vacía cuando el usuario quiere retirar y la estación llena cuando quiere devolver. Los dos se resuelven con el traslado de bicis que hace el operario.

3.1 Qué dato se registra en cada evento y en qué entidad

Evento	Actor	Dato que se registra o se consulta	Entidad del MER
Elige la estación de origen	Usuario	Consulta: estado y estación de cada bici	BICICLETA
Valida usuario y tarjeta	Sistema	Consulta: dni y vencimiento de la tarjeta	USUARIO, TARJETA
Crea el viaje y desbloquea la bici	Sistema	dni, numero_serie, estación de origen, id_tarjeta, fecha_hora_inicio; estado = EN_USO	VIAJE, BICICLETA
¿Hay anclaje libre?	Estación	Consulta: cantidad de bicis contra capacidad_max	ESTACION, BICICLETA
Cierra el viaje y cobra	Sistema	fecha_hora_fin y estación de destino; la bici pasa a la estación de destino	VIAJE, BICICLETA
Repara la bici en la estación	Operario	fecha, descripción, estación y legajo; estado = EN_MANTENIMIENTO y después DISPONIBLE	MANTENIMIENTO, BICICLETA
Traslada bicis entre estaciones	Operario	Nueva estación de cada bici	BICICLETA (el historial de traslados lo agregué en la Etapa 2)

Corrección del MER: el evento “cobra” necesitaba saber con qué tarjeta se paga, entonces agregué la relación “se paga con” entre VIAJE y TARJETA.

3.2 Cómo pasa este diagrama al manual de procedimientos

El diagrama de la Figura 2 va a ser la Figura 1 del manual (MP-01 “Alquiler, devolución y mantenimiento de bicicletas”), siguiendo la estructura de la Clase 7:

- **Objetivo:** que haya bicis disponibles y que cada viaje quede registrado.
- **Alcance:** desde que el usuario pide una bici hasta que queda anclada y disponible.
- **Definiciones:** estación, anclaje, viaje, reposicionamiento.
- **Responsables:** los cuatro carriles.
- **Descripción paso a paso:** un paso numerado por cada caja del diagrama, en el mismo orden y con las ramas Sí/No.
- **Registros:** las tablas del MER.
- **Indicadores:** por ejemplo, el porcentaje de intentos de retiro con la estación vacía (meta: menos del 5 %) y el tiempo promedio de reparación.
- **Control de cambios.**

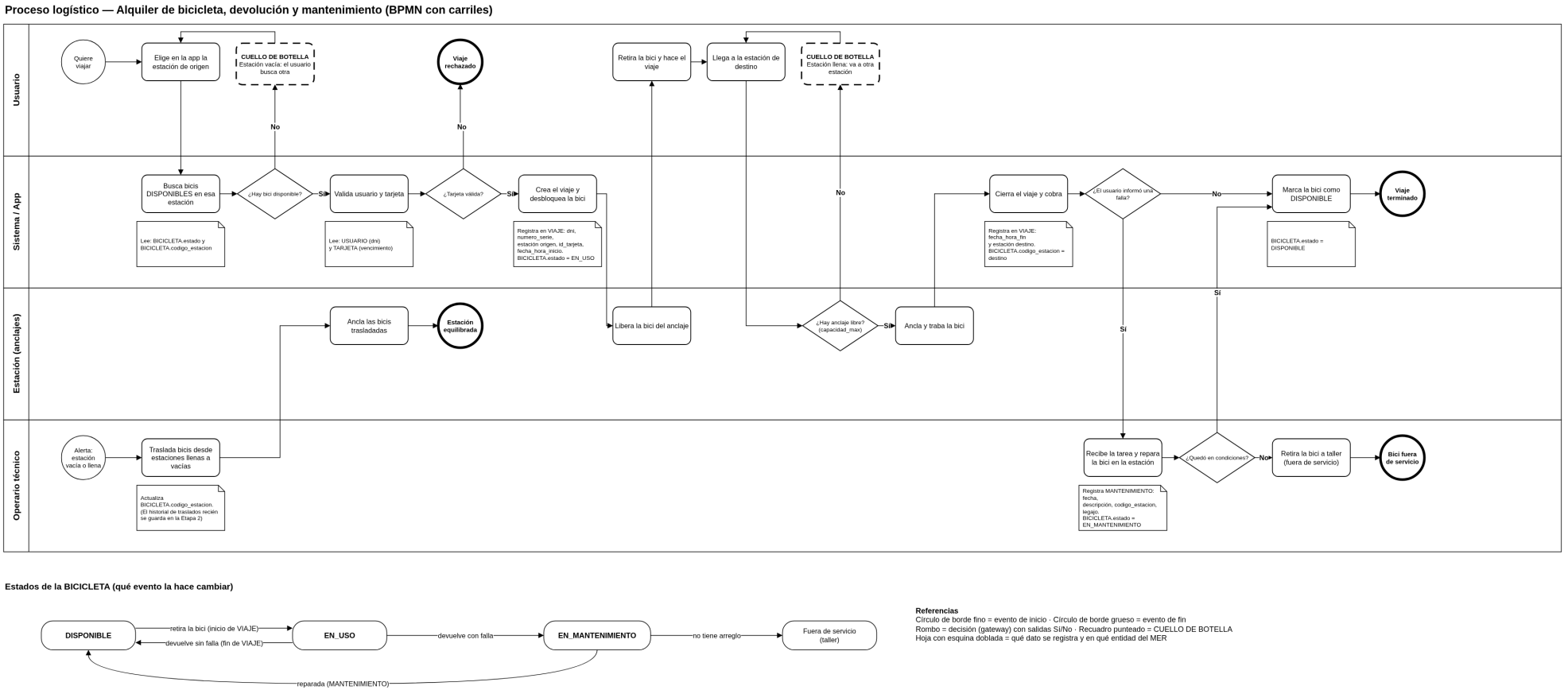


Figura 2. Proceso logístico en BPMN con carriles (archivo editable: bicicletas_BPMN.drawio)

4. MR y SQL con IA

4.1 Pedido (prompt) que usé

Sos un diseñador de bases de datos. Te paso un enunciado y mi modelo entidad-relación. Transformalo al modelo relacional (tablas, clave primaria y claves foráneas) usando las reglas clásicas: entidad fuerte → tabla; entidad débil → tabla con PK = PK de las dueñas + clave parcial; relación 1:N → FK del lado N (NOT NULL si la participación es total). Después escribí el script SQL completo para PostgreSQL (Supabase), con CHECK para los estados permitidos y para las reglas del enunciado, y datos de prueba ficticios. Enunciado: [enunciado completo del profesor]. Mi MER: ESTACION(codigo PK, direccion, capacidad_max); BICICLETA(numero_serie PK, estado) N:1 ESTACION; USUARIO(dni PK, nombre); TARJETA(id, marca, ultimos_4_digitos, vencimiento) N:1 USUARIO; VIAJE débil de USUARIO y BICICLETA (clave parcial fecha_hora_inicio;

fecha_hora_fin), N:1 estación origen, N:1 estación destino (opcional mientras está en curso), N:1 TARJETA; MANTENIMIENTO(id, fecha, descripcion) N:1 ESTACION, N:1 OPERARIO; OPERARIO(legajo PK, nombre). Reglas: una estación no puede superar su capacidad; un viaje termina después de empezar; una bici no puede estar en dos viajes abiertos; el viaje se paga con una tarjeta del mismo usuario.

4.2 Respuesta de la IA

Aclaración: para preparar este trabajo usé un asistente de IA (Grok Bot), y la respuesta que copio acá es la que me dio con ese pedido. [Si uso otra IA de la lista de la consigna (ChatGPT, DeepSeek, Gemini o el asistente de Supabase), reemplazo esta respuesta por la suya completa y repito la revisión del punto 4.3.]

Modelo relacional (MR):

- ESTACION (**codigo** PK, direccion, capacidad_max)
- BICICLETA (**numero_serie** PK, estado, codigo_estacion FK→ESTACION, obligatoria)
- USUARIO (**dni** PK, nombre)
- TARJETA (**id_tarjeta** PK, dni_usuario FK→USUARIO, obligatoria; marca, ultimos_4_digitos, vencimiento)
- OPERARIO (**legajo** PK, nombre)
- VIAJE (dni_usuario FK→USUARIO, numero_serie FK→BICICLETA, fecha_hora_inicio, fecha_hora_fin, codigo_estacion_origen FK→ESTACION obligatoria, codigo_estacion_destino FK→ESTACION opcional, id_tarjeta FK→TARJETA obligatoria; **PK = (dni_usuario, numero_serie, fecha_hora_inicio)**)
- MANTENIMIENTO (**id_mantenimiento** PK, fecha, descripcion, codigo_estacion FK→ESTACION obligatoria, legajo_operario FK→OPERARIO obligatoria)

El script SQL completo está en el Anexo A (sql/etapa1_crear_base.sql), y los datos de prueba en el Anexo B (sql/etapa1_datos_y_pruebas.sql).

4.3 Revisión crítica

En qué coincide con mi MER: las 7 entidades pasaron a 7 tablas. VIAJE tiene la clave compuesta que corresponde a una entidad débil. Cada relación 1:N quedó como clave foránea del lado N. Las participaciones obligatorias quedaron NOT NULL, y el destino del viaje quedó opcional, como yo había decidido.

Diferencias que encontré:

1. TARJETA y MANTENIMIENTO no tenían un identificador en el enunciado, así que la IA usó un número automático. Me parece bien.
2. Para que la tarjeta sea del mismo usuario, la IA armó una clave foránea doble (id_tarjeta, dni_usuario). Es más complicado de lo que yo esperaba, pero cumple la regla, así que lo dejé.
3. La base no obliga a que el usuario tenga al menos una tarjeta, y en el MER yo puse (1,N). Esto queda para la aplicación.
4. La capacidad de la estación no se puede controlar con un CHECK. La IA usó un trigger (una función que se ejecuta sola cada vez que se carga una bici). Lo probé y funciona.
5. Las tablas quedaron sin RLS (seguridad por filas), y Supabase lo avisa. Queda para resolver en la Etapa 2, cuando haga la aplicación.

6. Las fechas se guardan con zona horaria y Supabase las muestra en UTC (3 horas más que en Argentina). No es un error.
7. Lo comparé con el archivo bicicletas_compartidas.sql que pasó el profesor, que tiene solo 3 tablas:
   - usa un id numérico como clave, en vez del código de la estación y el número de serie de la bici;
   - la estación no tiene código y la bicicleta no tiene número de serie;
   - tiene un estado SIN_EXISTENCIA que el enunciado no nombra;
   - el operario tiene apellido;
   - no tiene USUARIO, TARJETA, VIAJE ni MANTENIMIENTO.

   En lo que tienen en común coincide con mi script (la bici con estación obligatoria y los estados controlados). Yo mantuve los identificadores del enunciado. [Consultar al profesor si prefiere usar id numéricos.]

5. Supabase

Creé el proyecto bicicletas-compartidas en mi cuenta personal de Supabase (región sa-east-1, São Paulo) y seguí estos pasos:

Paso 1. Abrí el proyecto.



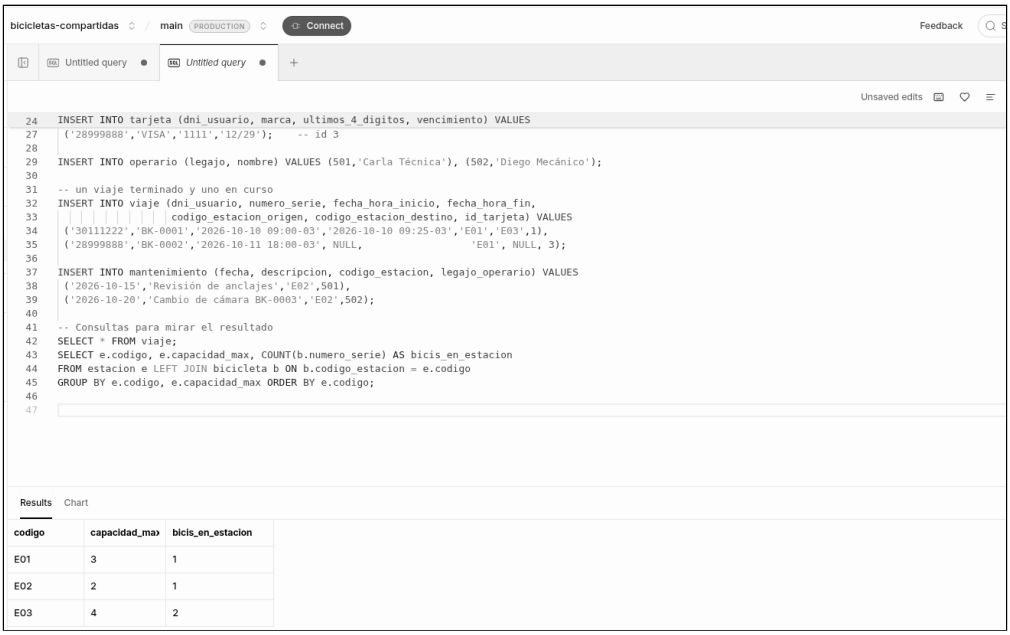
Captura 1. Proyecto bicicletas-compartidas en Supabase.

Paso 2. En SQL Editor, New query, pegué etapa1_crear_base.sql y lo ejecuté con Run. El resultado fue “Success. No rows returned”.



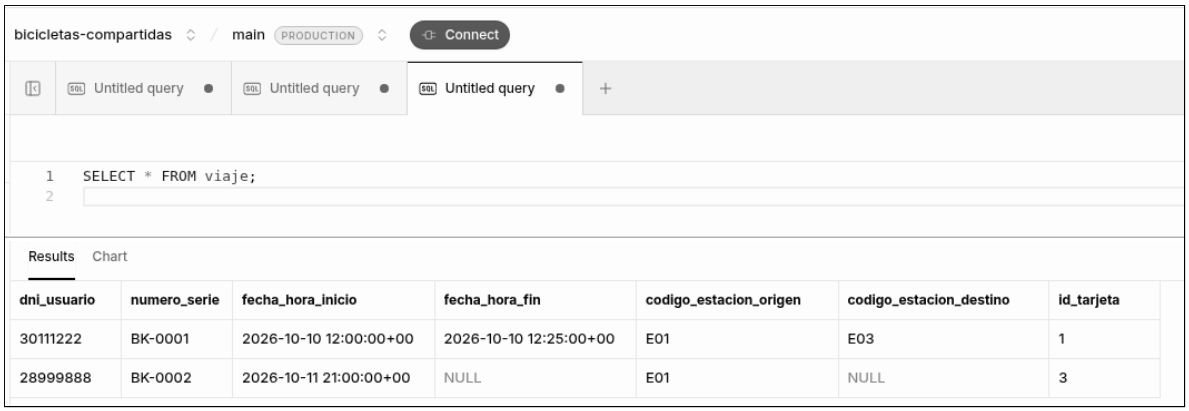
Captura 2. Script de creación ejecutado sin errores.

Paso 3. Cargué los datos ficticios (Parte A del archivo de pruebas). La consulta final muestra cuántas bicis tiene cada estación.



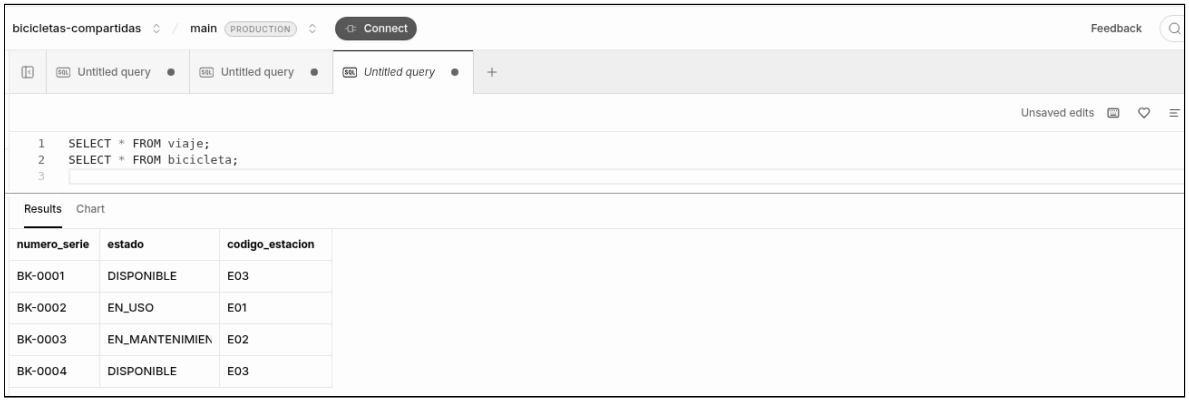
Captura 3. Datos de prueba cargados y bicis por estación.

Paso 4. Consulté las tablas viaje (2 filas: un viaje terminado y uno en curso, con destino y fin en NULL) y bicicleta (4 filas).



dni_usuario	numero_serie	fecha_hora_inicio	fecha_hora_fin	codigo_estacion_origen	codigo_estacion_destino	id_tarjeta
30111222	BK-0001	2026-10-10 12:00:00+00	2026-10-10 12:25:00+00	E01	E03	1
28999888	BK-0002	2026-10-11 21:00:00+00	NULL	E01	NULL	3

Captura 4. Contenido de la tabla viaje. Las horas se ven en UTC (3 horas más que en Argentina).



numero_serie	estado	codigo_estacion
BK-0001	DISPONIBLE	E03
BK-0002	EN_USO	E01
BK-0003	EN_MANTENIMIEN	E02
BK-0004	DISPONIBLE	E03

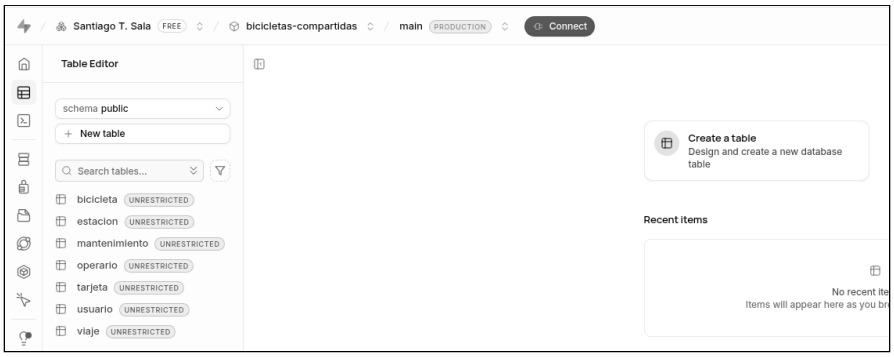
Captura 5. Contenido de la tabla bicicleta, con el estado y la estación de cada una.

Paso 5. Ejecuté la prueba B1 (bici con estado “ROTA”). Supabase la rechazó por la regla bicicleta_estado_check, que es lo que tenía que pasar.



Captura 6. Prueba B1 rechazada por la regla de estados permitidos.

Paso 6. En Table Editor aparecen las 7 tablas.



Captura 7. Las 7 tablas creadas en el Table Editor.

Errores y correcciones. Antes de pasar el script a Supabase, lo probé en un PostgreSQL 17 instalado en la computadora (Supabase también usa PostgreSQL) y corrió sin errores. Revisando los datos de ejemplo encontré dos inconsistencias: una bici figuraba DISPONIBLE pero estaba en un viaje en curso, y otra había terminado un viaje en E03 pero seguía asignada a E01. Las corregí para que coincidan con el diagrama logístico. En Supabase el script también corrió sin errores. Supabase avisó que las tablas quedaron sin RLS (seguridad por filas; por eso figuran como “UNRESTRICTED” en la Captura 7). Para esta etapa con datos ficticios no es un problema, pero lo voy a resolver en la Etapa 2, cuando la aplicación se conecte a la base.

Pruebas realizadas. Todas las corrí primero en PostgreSQL local. En Supabase ya corrí la carga de datos (A) y la prueba B1, con el mismo resultado; las demás las voy a repetir ahí.

Prueba	Qué intenté	Lo que tenía que pasar	Resultado
A	Cargar 3 estaciones, 4 bicis, 2 usuarios, 3 tarjetas, 2 viajes y 2 mantenimientos	Que lo acepte	Lo aceptó (también en Supabase)
B1	Bici con estado “ROTA”	Que lo rechace	Lo rechazó (también en Supabase, Captura 6)
B2	Bici en una estación que no existe (E99)	Que lo rechace	Lo rechazó
B3	Superar la capacidad de E02	Que lo rechace	Lo rechazó
B4	Viaje de un DNI que no existe	Que lo rechace	Lo rechazó
B5	Viaje que termina antes de empezar	Que lo rechace	Lo rechazó
B6	Pagar con la tarjeta de otro usuario	Que lo rechace	Lo rechazó
B7	La misma bici en dos viajes abiertos	Que lo rechace	Lo rechazó
B8	Mantenimiento sin operario	Que lo rechace	Lo rechazó
B9	DNI con letras	Que lo rechace	Lo rechazó
B10	Borrar una estación que tiene bicis	Que lo rechace	Lo rechazó

Anexo A — sql/etapa1_crear_base.sql

```
-- ======================================================================
-- Bicicletas compartidas - ETAPA 1 - Script de creación (PostgreSQL / Supabase)
-- Correr completo en Supabase > SQL Editor > New query > Run.
-- Solo datos ficticios. Se puede volver a correr: primero borra las tablas.
-- ======================================================================
```

```
DROP TABLE IF EXISTS viaje, mantenimiento, tarjeta, bicicleta, operario, usuario, estacion
CASCADE;
DROP FUNCTION IF EXISTS controlar_capacidad_estacion() CASCADE;

-- ESTACION (entidad fuerte)
CREATE TABLE estacion (
  codigo         VARCHAR(10)  PRIMARY KEY,
  direccion      VARCHAR(150) NOT NULL,
  capacidad_max  INTEGER      NOT NULL CHECK (capacidad_max > 0)
);

-- BICICLETA (fuerte). N:1 con ESTACION -> FK en bicicleta (lado N), obligatoria
CREATE TABLE bicicleta (
  numero_serie     VARCHAR(30) PRIMARY KEY,
  estado           VARCHAR(20) NOT NULL DEFAULT 'DISPONIBLE'
                   CHECK (estado IN ('DISPONIBLE','EN_USO','EN_MANTENIMIENTO')),
  codigo_estacion  VARCHAR(10) NOT NULL REFERENCES estacion(codigo)
);

-- USUARIO (fuerte)
CREATE TABLE usuario (
  dni     VARCHAR(8)   PRIMARY KEY CHECK (dni ~ '^[0-9]{7,8}$'),
  nombre  VARCHAR(120) NOT NULL
);

-- TARJETA (medio de pago). N:1 con USUARIO. Nunca se guarda el número completo.
CREATE TABLE tarjeta (
  id_tarjeta         INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  dni_usuario        VARCHAR(8) NOT NULL REFERENCES usuario(dni) ON DELETE CASCADE,
  marca              VARCHAR(30) NOT NULL,
  ultimos_4_digitos  CHAR(4)    NOT NULL CHECK (ultimos_4_digitos ~ '^[0-9]{4}$'),
  vencimiento        CHAR(5)    NOT NULL CHECK (vencimiento ~ '^(0[1-9]|1[0-2])/[0-9]{2}$'),
  UNIQUE (id_tarjeta, dni_usuario)   -- permite exigir que el viaje se pague con una tarjeta del
mismo usuario
);

-- OPERARIO TÉCNICO (fuerte)
CREATE TABLE operario (
  legajo  INTEGER      PRIMARY KEY CHECK (legajo > 0),
  nombre  VARCHAR(120) NOT NULL
);

-- VIAJE (entidad débil de USUARIO y BICICLETA)
-- PK = PK de las dueñas (dni_usuario, numero_serie) + clave parcial (fecha_hora_inicio)
CREATE TABLE viaje (
  dni_usuario              VARCHAR(8)  NOT NULL REFERENCES usuario(dni),
  numero_serie             VARCHAR(30) NOT NULL REFERENCES bicicleta(numero_serie),
  fecha_hora_inicio        TIMESTAMPTZ NOT NULL,
  fecha_hora_fin           TIMESTAMPTZ,                        -- NULL = viaje en curso
  codigo_estacion_origen   VARCHAR(10) NOT NULL REFERENCES estacion(codigo),
  codigo_estacion_destino  VARCHAR(10)          REFERENCES estacion(codigo), -- NULL mientras
está en curso
  id_tarjeta               INTEGER     NOT NULL,
  PRIMARY KEY (dni_usuario, numero_serie, fecha_hora_inicio),
  FOREIGN KEY (id_tarjeta, dni_usuario) REFERENCES tarjeta(id_tarjeta, dni_usuario),
  CHECK (fecha_hora_fin IS NULL OR fecha_hora_fin > fecha_hora_inicio),
  CHECK ((fecha_hora_fin IS NULL) = (codigo_estacion_destino IS NULL))
);
-- Una bicicleta no puede estar en dos viajes abiertos a la vez, ni un usuario tampoco
CREATE UNIQUE INDEX un_viaje_abierto_por_bici    ON viaje (numero_serie) WHERE fecha_hora_fin IS
NULL;
CREATE UNIQUE INDEX un_viaje_abierto_por_usuario ON viaje (dni_usuario)  WHERE fecha_hora_fin IS
NULL;

-- MANTENIMIENTO (tarea programada). N:1 con ESTACION y N:1 con OPERARIO, ambas obligatorias
CREATE TABLE mantenimiento (
  id_mantenimiento  INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  fecha             DATE    NOT NULL,
  descripcion       TEXT    NOT NULL,
  codigo_estacion   VARCHAR(10) NOT NULL REFERENCES estacion(codigo),
  legajo_operario   INTEGER     NOT NULL REFERENCES operario(legajo)
);

-- Regla: una estación no puede tener más bicicletas que su capacidad_max
CREATE FUNCTION controlar_capacidad_estacion() RETURNS trigger AS $$
BEGIN
```

```
  IF (SELECT COUNT(*) FROM bicicleta WHERE codigo_estacion = NEW.codigo_estacion)
     > (SELECT capacidad_max FROM estacion WHERE codigo = NEW.codigo_estacion) THEN
    RAISE EXCEPTION 'La estacion % supera su capacidad maxima', NEW.codigo_estacion;
  END IF;
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_capacidad_estacion
AFTER INSERT OR UPDATE OF codigo_estacion ON bicicleta
FOR EACH ROW EXECUTE FUNCTION controlar_capacidad_estacion();
```

Anexo B — sql/etapa1_datos_y_pruebas.sql

```
-- ======================================================================
-- ETAPA 1 - Datos ficticios (PARTE A) y pruebas (PARTE B)
-- PARTE A: correr toda junta. Tiene que terminar "Success".
-- PARTE B: correr cada prueba POR SEPARADO (seleccionar la línea y Run).
--          Cada una TIENE QUE DAR ERROR: eso demuestra que la base respeta las reglas.
-- ======================================================================

-- ---------- PARTE A: datos válidos ----------
INSERT INTO estacion (codigo, direccion, capacidad_max) VALUES
 ('E01','Plaza Central 100', 3),
 ('E02','Av. Costanera 450', 2),
 ('E03','Terminal de Ómnibus', 4);

INSERT INTO bicicleta (numero_serie, estado, codigo_estacion) VALUES
 ('BK-0001','DISPONIBLE','E03'),   -- terminó un viaje en E03
 ('BK-0002','EN_USO','E01'),       -- está en un viaje en curso que salió de E01
 ('BK-0003','EN_MANTENIMIENTO','E02'),
 ('BK-0004','DISPONIBLE','E03');

INSERT INTO usuario (dni, nombre) VALUES
 ('30111222','Ana Ficticia'),
 ('28999888','Bruno Inventado');

INSERT INTO tarjeta (dni_usuario, marca, ultimos_4_digitos, vencimiento) VALUES
 ('30111222','VISA','4242','08/28'),    -- id 1
 ('30111222','MASTERCARD','5555','01/27'), -- id 2
 ('28999888','VISA','1111','12/29');    -- id 3

INSERT INTO operario (legajo, nombre) VALUES (501,'Carla Técnica'), (502,'Diego Mecánico');

-- un viaje terminado y uno en curso
INSERT INTO viaje (dni_usuario, numero_serie, fecha_hora_inicio, fecha_hora_fin,
                   codigo_estacion_origen, codigo_estacion_destino, id_tarjeta) VALUES
 ('30111222','BK-0001','2026-10-10 09:00-03','2026-10-10 09:25-03','E01','E03',1),
 ('28999888','BK-0002','2026-10-11 18:00-03', NULL,                  'E01', NULL, 3);

INSERT INTO mantenimiento (fecha, descripcion, codigo_estacion, legajo_operario) VALUES
 ('2026-10-15','Revisión de anclajes','E02',501),
 ('2026-10-20','Cambio de cámara BK-0003','E02',502);

-- Consultas para mirar el resultado
SELECT * FROM viaje;
SELECT e.codigo, e.capacidad_max, COUNT(b.numero_serie) AS bicis_en_estacion
FROM estacion e LEFT JOIN bicicleta b ON b.codigo_estacion = e.codigo
GROUP BY e.codigo, e.capacidad_max ORDER BY e.codigo;

-- ---------- PARTE B: pruebas que DEBEN FALLAR (una por vez) ----------
-- B1 estado no permitido
INSERT INTO bicicleta VALUES ('BK-0099','ROTA','E01');
-- B2 estación inexistente (integridad referencial)
INSERT INTO bicicleta VALUES ('BK-0098','DISPONIBLE','E99');
-- B3 supera capacidad (E02 tiene capacidad 2 y ya tiene 1: no entran 2 más)
INSERT INTO bicicleta VALUES ('BK-0097','DISPONIBLE','E02'), ('BK-0096','DISPONIBLE','E02');
-- B4 viaje sin usuario (no existe el DNI): un viaje no existe sin el usuario
INSERT INTO viaje VALUES ('99999999','BK-0004','2026-10-11 10:00-03',NULL,'E03',NULL,1);
-- B5 viaje con fin anterior al inicio
INSERT INTO viaje VALUES ('30111222','BK-0004','2026-10-11 10:00-03','2026-10-11
09:00-03','E03','E01',1);
-- B6 pagar con la tarjeta de OTRO usuario (tarjeta 3 es de Bruno)
INSERT INTO viaje VALUES ('30111222','BK-0004','2026-10-11 11:00-03',NULL,'E03',NULL,3);
-- B7 la bici BK-0002 ya está en un viaje abierto
```

```
INSERT INTO viaje VALUES ('30111222','BK-0002','2026-10-11 18:30-03',NULL,'E01',NULL,1);
-- B8 mantenimiento sin operario
INSERT INTO mantenimiento (fecha, descripcion, codigo_estacion, legajo_operario) VALUES
('2026-10-30','Limpieza','E01',NULL);
-- B9 DNI con letras
INSERT INTO usuario VALUES ('12AB5678','Prueba Mal');
-- B10 borrar una estación que tiene bicicletas (no se permite dejar bicis "huérfanas")
DELETE FROM estacion WHERE codigo = 'E01';
```